

TripMate: A Tool-Augmented Travel-Planning Agent

Phase 1: Problem statement and proposed architecture

Group No.: _____ · Course: Generative AI (PSIS activity) · Activity 2: Tool-Augmented / Agentic LLM Application
Submission date: 19 September 2026

1. Selected activity and application

Activity 2: Tool-augmented / agentic LLM application. Application: **Travel Planner** ("TripMate"), a chat assistant that plans short trips by calling external APIs as tools.

2. Problem statement

Planning even a weekend trip means combining facts that change every day (weather forecast, exchange rates, public holidays) with destination knowledge (sights, food, transport) and a budget. A stand-alone LLM cannot know next week's forecast or today's exchange rate. Asked anyway, it produces plausible but unsourced numbers, and it often gets multi-item budget arithmetic wrong. We will build a conversational agent where **the LLM plans and explains, but every time-sensitive fact comes from a tool call**. The agent must remember the conversation, return a structured itinerary, and behave safely when a tool fails or returns something unexpected, without inventing the missing data.

3. Objectives

- Answer travel questions and produce day-by-day plans grounded in live tool results, citing the source.
- Integrate at least five LangChain components (prompt templates, tools + agent, memory, output parser, LCEL).
- Detect tool failures (timeouts, rate limits, empty or unexpected responses, unknown places) and report them honestly to the user.
- Evaluate on at least five test scenarios, including failure cases, and examine answer grounding critically.

4. Proposed architecture

Flow: user (Streamlit chat / notebook) → **LangChain agent** (Gemini chat model with a system `PromptTemplate` and conversation memory) ⇌ **tools** → free public APIs → grounded answer → **LCEL chain** `prompt | LLM | PydanticOutputParser` → validated JSON itinerary.

LangChain component	Planned use	
PromptTemplate / ChatPromptTemplate	System prompt with today's date and grounding rules; itinerary-formatting prompt	
Tools + Agent (<code>create_agent</code>)	The LLM decides which tools to call, in which order, and when to stop	
Memory (LangGraph checkpointer)	Follow-up questions reuse the destination, dates and earlier tool results	
OutputParser (<code>PydanticOutputParser</code>)	Validated JSON trip plan: days, weather, budget, caveats	
LCEL composition	<code>prompt llm parser</code> , plus a fallback chain that repairs invalid JSON	

Tool	External API (free, no key)	Purpose
<code>get_weather</code>	Open-Meteo Geocoding + Forecast	Daily forecast for the trip dates (16-day horizon)
<code>convert_currency</code>	Frankfurter (European Central Bank rates)	Budgets and amounts in the traveller's currency
<code>get_public_holidays</code>	Nager.Date	Crowds, closures and price peaks during the trip
<code>get_destination_guide</code>	Wikivoyage (MediaWiki API)	What to see and do, food, getting around
<code>estimate_trip_budget</code>	Local Python function	Exact budget arithmetic from the LLM's unit-cost estimates

Tech stack: Python 3.12, LangChain 1.x + LangGraph, Google Gemini (free tier), requests, Pydantic, Streamlit, pytest.

Deliverables by phase: Phase 2 (23 Sep): working agent with weather, currency and guide tools in a notebook. Phase 3 (30 Sep): all tools, memory, output parser, failure handling, evaluation, report and video.

5. Team members and roles

Member	Role	Responsibilities
Member 1: <name, roll no.>	Agent & LangChain integration lead	Agent design, system prompt, memory, output parser and LCEL chains
Member 2: <name, roll no.>	Tools & API integration	Tool functions, API clients, error handling, caching
Member 3: <name, roll no.>	Testing & evaluation	Test scenarios, unit tests, evaluation of grounding and failures
Member 4: <name, roll no.>	UI, documentation & demo video	Streamlit UI, architecture diagram, README, report, video